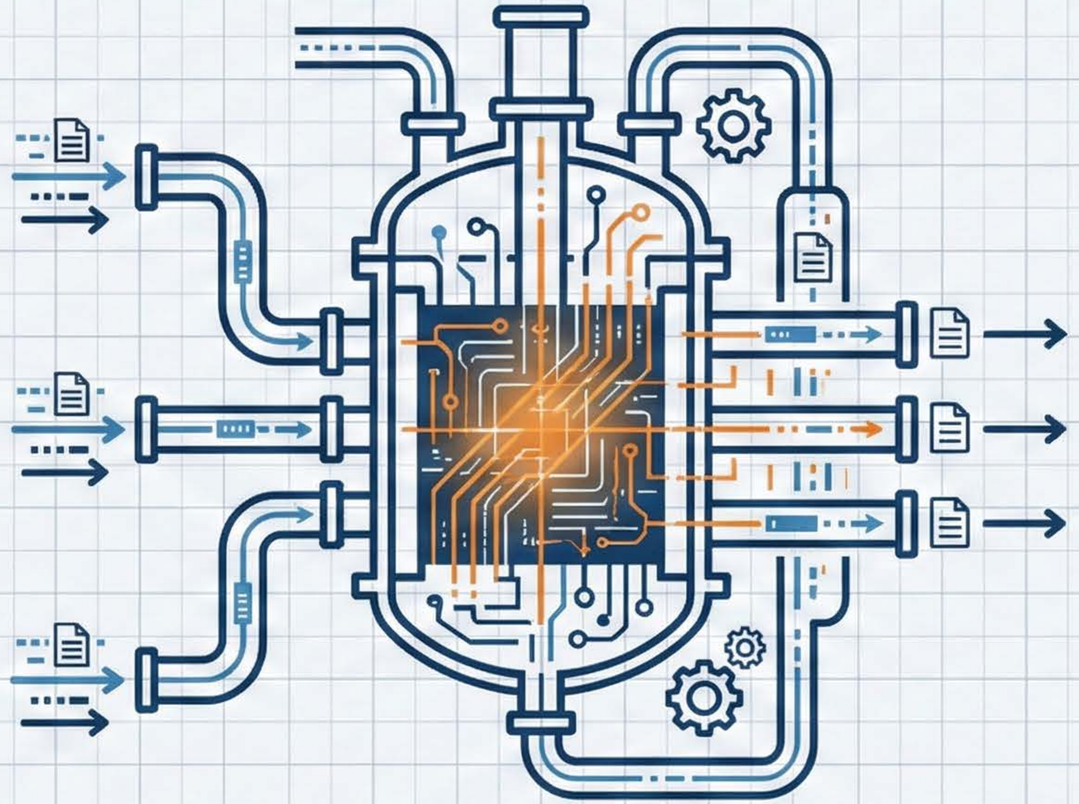
NumPy 陣列運算

向量化運算的核心機制

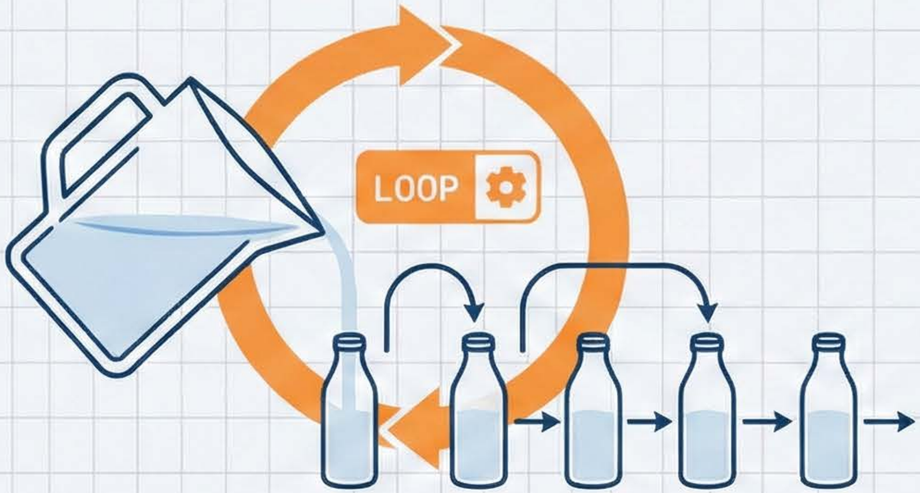


單元: Unit 03 | 應用領域: 化工與數據科學

NotebookLM

為什麼需要向量化？從「逐一填充」到「並行處理」

Python List (傳統迴圈)

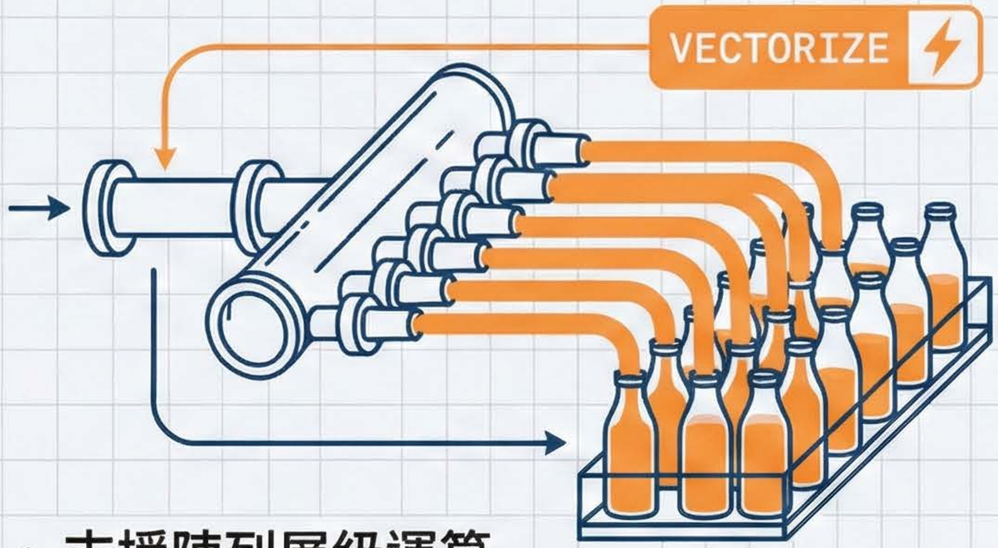


- 需要明確撰寫迴圈

機制：逐一讀取元素 -> 計算 -> 儲存

狀態：低效能

NumPy Array (向量化)



- 支援陣列層級運算

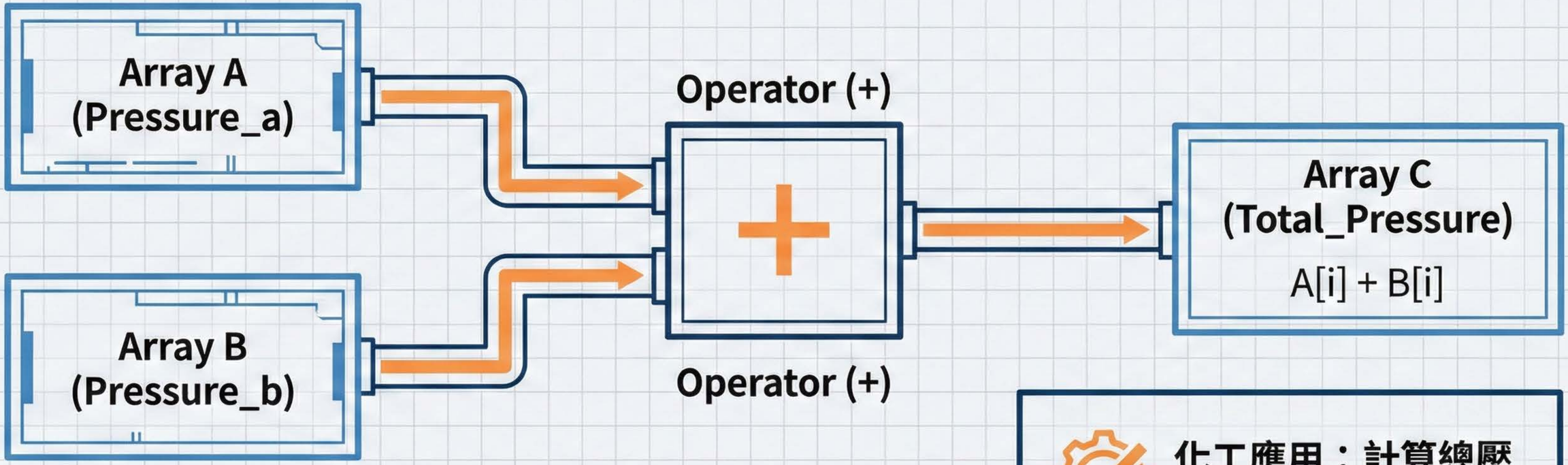
機制：元素層級運算 (SIMD)

狀態：高效能 (快 10-100 倍)

在 NumPy 數位工廠中，我們不處理單個樣品，我們同時處理整批批次。

NotebookLM

4.1 基本算術運算：元素層級的平行處理




```
pressure_a = np.array([1.2, 1.5, 1.8])
pressure_b = np.array([0.1, 0.2, 0.1])
total_pressure = pressure_a + pressure_b
# Result: [1.3, 1.7, 1.9]
```

化工應用：計算總壓 (Dalton's Law)

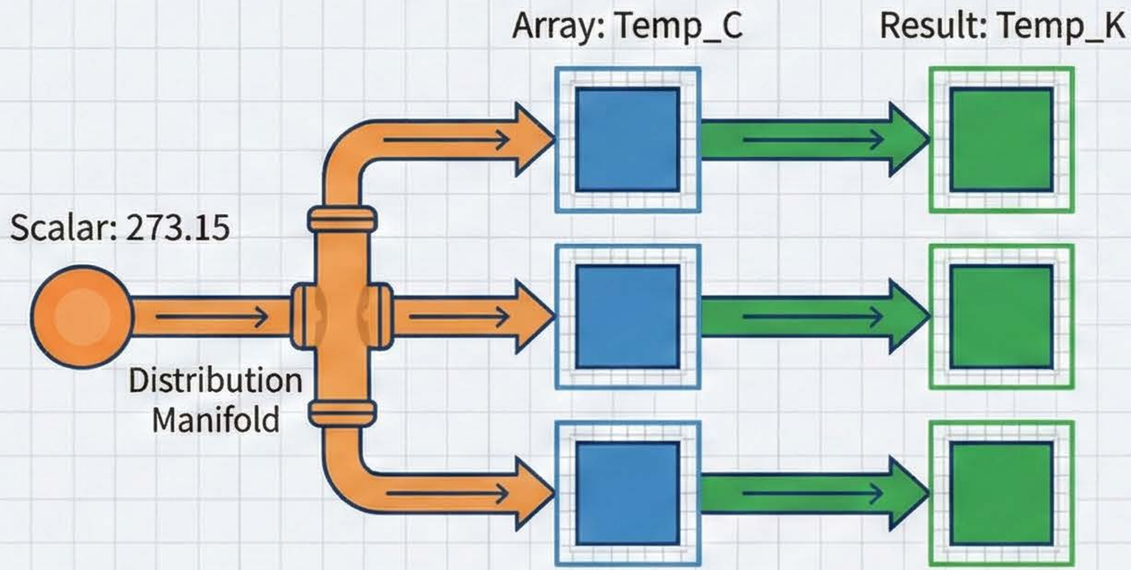
$$P_{total} = P_a + P_b$$

支援運算子：+, -, *, /, **

NotebockLM

純量運算與廣播機制

Broadcasting



定義

當陣列與單一數值(Scalar)運算時，該數值會自動擴展(Broadcast)至整個陣列大小。

化工應用：單位換算

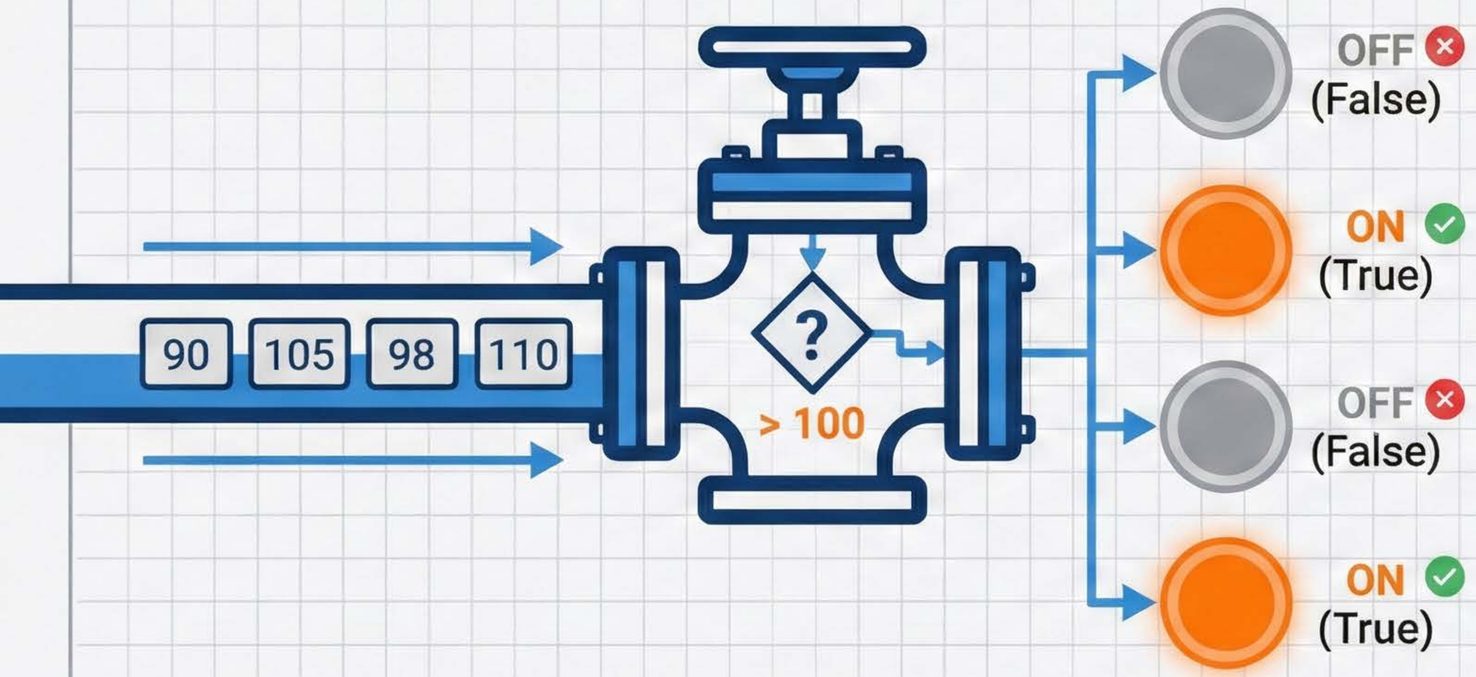
攝氏轉克氏 ($T_K = T_C + 273.15$)

```
temp_c = np.array([25, 100, 150])
temp_k = temp_c + 273.15 # 自動廣播
# Result: [298.15, 373.15, 423.15]
```

無需編寫迴圈讓每個數字加 273.15，NumPy 自動完成。

NotebookLM

4.2 比較運算：數位工廠的感測與監控



比較運算的結果不是數值，而是**布林陣列 (Boolean Array)**。

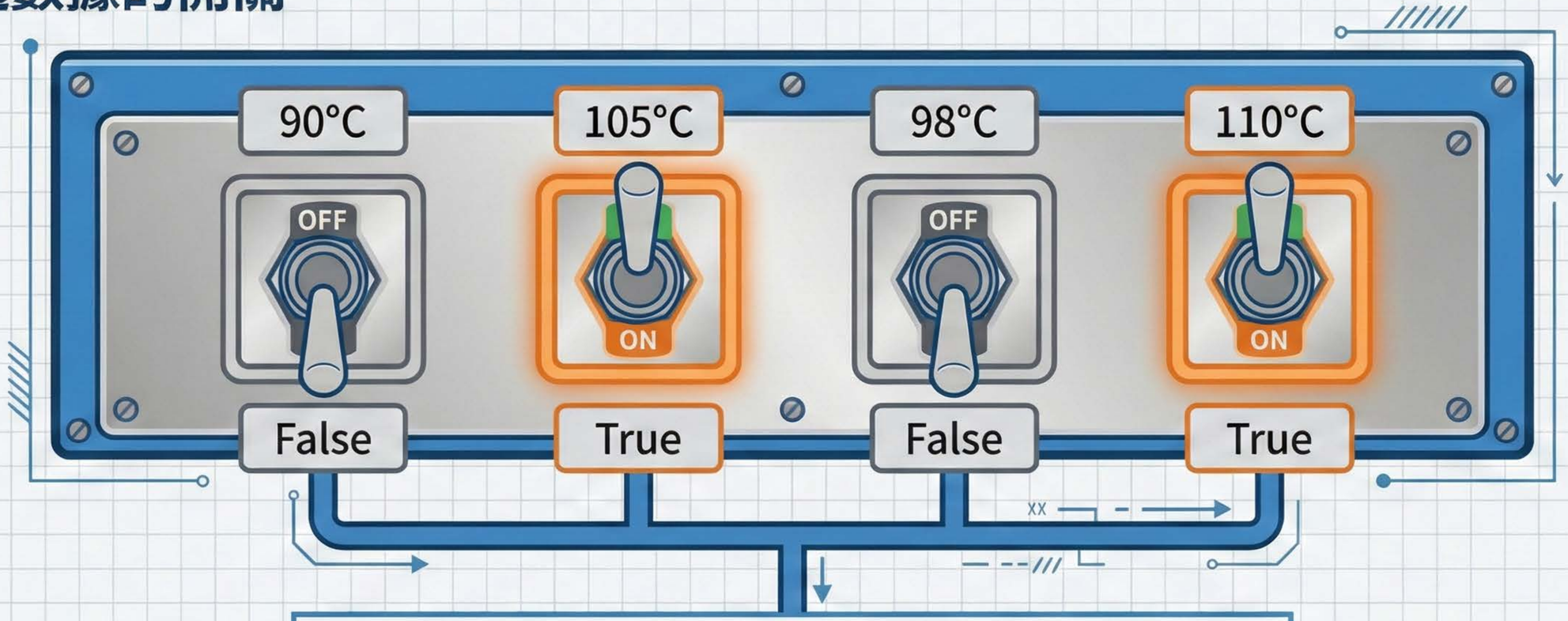
== (等於)
!= (不等於)
> (大於)
< (小於)

```
# 化工應用：反應器過熱監控
temp_reactor = np.array([90, 105, 98, 110])
is_overheated = temp_reactor > 100
# Result: [False, True, False, True]
```

NotebookLM

視覺化布林遮罩 (Boolean Masks)

篩選數據的開關



這些布林值 (True/False) 可用於後續的「布林索引」(Boolean Indexing) 來自動篩選出異常數據。

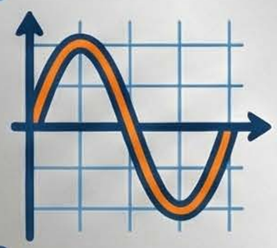
NotebookLM

4.3 通用函式 (Universal Functions, ufunc)

高效能的數學運算反應器

NumPy 提供大量快速的元素層級數學函式，底層以 C 語言優化，速度遠快於 Python 內建 math 模組。

三角函數 (Trigonometry)



```
np.sin()
np.cos()
np.tan()
```

應用

波動分析

指數與對數 (Exponentials)

```
np.exp()
np.log()
np.log10()
```

應用

反應動力學

其他運算 (Roots & Abs)

```
np.sqrt()
np.abs()
```

應用

幾何計算

⚠ **注意：對陣列運算時，請務必使用 np.sin() 而非 math.sin()。**

NotebookLM

實務應用：阿瑞尼斯方程式 (Arrhenius Equation)

阿瑞尼斯新方程式

$$k = A \cdot \exp\left(\frac{-E_a}{R \cdot T}\right)$$

k = 反應速率常數
E_a = 活化能
T = 溫度陣列

```
1. k = A * np.exp(-Ea / (R * T))
```

優勢：程式碼簡潔 (Concise)，易讀 (Readable)，運算高效 (Efficient)。
一行程式碼即可處理數百萬筆溫度數據。

NotebookLM

總結與最佳實踐 (Summary & Best Practices)

CHECKLIST

- ☑ **優先使用向量化 (Vectorization First)**
  Avoid loops. Think in arrays.
  (避免迴圈，以陣列思考)
- ☑ **理解運算類型 (Know Your Operations)**
  Arithmetic (+-*/) 改變數值，
  Comparison (><) 產生遮罩，Ufuncs
  (np.exp) 處理科學運算。
- ☑ **注意形狀 (Watch the Shapes)**
  善用廣播 (Broadcasting) 機制來簡
  化純量與陣列的運算。

Code is read much more often than it is written.
— Write clean, vectorized code.

NotebookLM